

Dependent Type Checking in Epilog

Building a Calculus of Constructions using Logic Programming

Bryce Goldman

1 Introduction

With the advent of LLMs as tools for code generation and mathematical research, automated theorem proving using interactive proof assistants has grown increasingly relevant in the domain of formal methods as a mechanism for verifying AI-generated results. Many modern proof assistants, such as Lean [10], Rocq [11], and Agda [4], are built on a foundation of dependent type theory (DTT) — a type theory [7] in which types are themselves terms, and which are allowed to range over other terms. This formalism is extremely powerful, and in fact it is generally undecidable [2] to statically type-check a dependently-typed program. However, using a slightly restricted variant of fully expressive DTT, called the calculus of constructions (CoC) [8], one can implement a dependently-typed functional programming language whose type-checking algorithm is equivalent to a proof-verifier.

Most existing CoC implementations are facilitated by either systems or functional languages; the former for performance reasons, and the latter because pattern-matching lends itself naturally to type-checking algorithms. I propose a third, largely unexplored option: logic programming [6]. In addition to benefiting from pattern-matching support, logic programming languages are natively equipped with powerful term unification algorithms. Universe polymorphism and type inference are key challenges in the design of a CoC system, and both obstructions essentially reduce to unifying some metavariable with the syntactic form of a ground term. Since logic programs naturally handle precisely this flavor of problem, my CoC system, built in Epilog [3], circumvents the need to implement these sophisticated unification algorithms entirely, vastly simplifying the typechecker. I argue that the logic programming paradigm canonically lends itself to the design patterns of a CoC typechecker.

2 Dependent Type Theory

A *type theory* [7] is a formal mechanism which describes the properties of expressions within some programming language. An expression x assigned a particular *type* A , denoted $x : A$, inherits the behavior or interface of A . For instance, a natural number $n : \mathbf{Nat}$ can be added or multiplied with other natural numbers, while a string $s : \mathbf{String}$ can be concatenated with other strings, or searched for particular substrings. The type of an expression serves as a compact description of how that expression can be used.

In practice, one often relies on a type system to prevent runtime errors, by catching problems statically at compile-time. For instance, the following program should be rejected, since the typechecker cannot prove that one natural number can be appended to another:

```
evil (n : Nat) (m : Nat) : Nat := append n m
```

However, some errors may still slip through the cracks of a type system. For instance, suppose `List A` denotes the type of lists of elements of some type A :

```
inductive List (A : Type) :=
  nil : List A
  cons : A -> List A -> List A

head : List A -> A
head (cons x xs) := x
```

with `nil` denoting the empty list, `cons x xs` denoting the list `xs` prepended with element $x : A$, and `headA` the first element of a list. Then

```
hmm (xs : List A) : A := head xs
also_evil : A := hmm nil
```

should not type-check,¹ but to do this we would need to encode the length of a list within its type. This naturally leads to the classical example of the `Vec` type, defined below:

```
inductive Vec (n : Nat) (A : Type) :=
  nil : Vec zero A
  cons : Vec n A -> A -> Vec (succ n) A
```

Here `Vec n A` denotes the type of all vectors of elements of A with length n , where `nil` is the empty vector (of length zero) and `cons` appends a given $x : A$ to a vector of length n to produce a new vector of length `succ n` $= n + 1$. Using this type in place of our previous list type, we can catch bugs like out-of-bounds accesses or reads from empty vectors simply by checking the type of a program.

Unlike `List`, the type constructor `Vec` *depends* on a term $n : \text{Nat}$. Thus, a type theory that permits types like `Vec` must accommodate types that depend on terms. In (Martin-Löf, or intuitionistic) *dependent type theory* (DTT) [1, 7], types are themselves terms of some type² `Type`, as suggested by the notation used above for A . DTT (and its variants) enjoy much more expressive power than non-dependent type theories, and consequently can detect issues in a program at a much more granular level. Unfortunately, this expressive power comes at a cost: dependently-typed programs are much harder to type-check, and in fact type-checking a general DTT program is undecidable [1]. Thus, we are forced to make certain concessions, outlined in Section 3.

In general, we can think of a type system as a means of proving guarantees about the behavior of a program. A stronger (i.e. more expressive) type system will be able to capture a broader range of behaviors, as seen above: if we limit ourselves to types like `List`, then we may not detect errors that would be caught using a more powerful type like `Vec`. This perspective can be made formal, via the Curry-Howard Isomorphism [5, 9], which establishes a one-to-one correspondence between types and theorems, and between terms (or programs) and proofs. A term (or program) $x : A$ can be viewed as a description for how to build a particular instance of type A , which, under this interpretation, is equivalent to instructions for how to prove A ; type-checking $x : A$ then becomes a verification that x proves A . This is precisely how modern proof assistant languages such as Lean, Rocq, and Agda mechanically validate proofs, by translating them into dependently-typed programs within the “calculus of constructions” and simply verifying that the resulting terms type-check.

3 The Calculus of Constructions

The *calculus of constructions* (CoC) [8] is a derivative of the more expressive Martin-Löf DTT. Classically, the latter enjoys an internalized form of equality, called *extensional equality*, which is the source of many of the obstructions to automating type resolution in a dependent theory. CoC simplifies type-checking by limiting judgmental equalities to only encompass declarations; that is, two terms are only considered judgmentally equal when they have the same syntactic definition. This makes the type resolution tractable for most of the dependently-typed programs one encounters in nature, at the expensive of requiring more work to “convert” between propositionally (but not judgmentally) equivalent terms.

Despite (usually) being decidable, type resolution in CoC is nontrivial due to the tight coupling of the type dynamics in a dependently-typed system. To elaborate the type of a term of *dependent function type* (see section 2.4.1 of [1]) $\Pi(x : A).B(x)$, for instance, one must infer the type of A , which itself may require resolving the types of terms that A depends on, then type-check $B(x)$ in the context $x : A$, and finally infer the type of $B(x)$.

¹One would typically wrap the codomain of `head` in a Maybe monad for a more robust definition, for exactly this reason.

²Readers who live in fear of Russell’s Paradox should at this point become anxious. We will later briefly outline how one avoids size issues using a hierarchy of universes.

Moreover, because it would be impractical to require the user to annotate every term in their program with a type, the types of many terms must be solved, by pattern-matching on the term’s syntactic structure. This can involve solving for metavariables — variables in the meta-theory which must be replaced with terms within the theory by the type elaborator — which utilizes a unification algorithm not unlike that of logic programs.

As noted in Section 2, since types themselves are considered terms in CoC, the type theory must support a “type of types,” typically denoted **Type**. However, naïvely incorporating a type of *all* types into a type theory quickly leads to size issues similar to Russell’s Paradox in set theory, so instead one must introduce a hierarchy of *type universes*

$$u_0 < u_1 < u_2 < \dots$$

Universe u_i is contained in u_j for each $j > i$, and a type which ranges over types in universe u_i must be contained in a larger universe (so that it cannot depend on itself). A persistent feature across CoC-based proof assistants is *universe polymorphism*, which allows a term to be parameterized by a variable universe, so long as all instances of that term can unify that variable with a particular, fixed universe level. For instance, our **List** definition above should really be

```
inductive List.{u} (A : Sort u) :=
  ...
```

where **u** is allowed to be resolved to any universe level, and **Sort u** is the type of all types in universe level **u** (which itself belongs to universe **u** + 1). Instantiating a particular list such as

```
nat_list : List Nat := cons zero nil
```

requires that the typechecker be able to solve **u** = 1, since **Nat** : **Sort** 1. (One often abbreviates **Type** := **Sort** 1 and **Prop** := **Sort** 0.) Similar to the metavariable solver for type inference, the universe solver for a CoC system is essentially just a structural unification algorithm, much like those employed by logic programming interpreters such as Epilog. This motivates my argument that a CoC typechecker naturally embeds in logic programming, which is supported by my implementation, described below in Section 4.

4 Design and Implementation

My implementation of a CoC typechecker is broken down into three main components:

1. A backend consisting entirely of Epilog rules, which performs the type-checking and type inference logic for CoC.
2. A Peggy parser, for lexing and parsing textual input into an intermediate representation which is compatible with the CoC Epilog rules.
3. A Javascript frontend, which parses the user’s input, calls Epilog to executes queries on the parsed IR, and displays the results to a simple webpage.

The bulk of the work on the project was writing, testing, and debugging the Epilog backend, so I elide the details of the other components (although the complete codebase can be viewed at this github repository; the Epilog ruleset can also be found in the appendix. The current deployment of the web application for the project can also be demoed here.

The Epilog rules are broken up into seven packages of rules (along with some additional miscellany):

Structure: defines the structure of the ground terms used to represent the objects of interest in my CoC implementation — terms, declarations, universes, patterns, and parameters — and predicates to check whether a given object belongs to a particular structure.

Universes: implements the interface for universe expressions, which are one of the following:

- A constant universe level: **uconst**(n) for some nonnegative n .

- A universe variable: `uvar(x)` for some string x . These must be replaced with fresh metavariables which are then unified at each instantiation of a term.
- An additive expression: `uadd(u, n)` for some universe expression u and nonnegative n . Represents the universe level n levels above u .
- A universe maximum: `umax(u, v)` for universe expressions u and v . Represents the smallest universe containing both u and v .

Includes protocols for normalizing universe expressions (e.g. reducing `uconst(n) + uconst(m)` to `uconst(m + n)`) and solving for universe metavariables.

Contexts: defines the map and context interfaces. Maps are implemented simply as lists of key-value pairs, with insertion, update, and lookup support. Contexts are fundamental to the execution of the typechecker, and store all the known type information (prior declarations, types of local variables, etc.) at a particular point while checking or inferring a type. The context data structure is just a collection of maps storing different sub-contexts.

Substitution: utilities pertaining to term (and universe) substitution, wherein a variable or subterm within some target term (or universe) must be consistently replaced by some other source term (or universe). For instance, this is needed to reduce a function application of some lambda abstraction term to an argument term:

$$(\lambda(x : A).e(x)) a \Rightarrow e(a)$$

This module also provides infrastructure for replacing syntactic variables (e.g. terms of the form `var("x")` with a metavariable `X`) which enables unification via Epilog’s native solver.

Equivalence: provides functions for term normalization (via β and η equivalence for lambda terms), α conversion of function types, and definitional equality checks. Also implements the main unification predicate, which dispatches to Epilog’s unifier to solve for any term or universe metavariables.

Type Checking: implements the main methods for checking that each declaration in the input module is well-typed, and that all metavariables can be resolved to concrete terms/universe levels. Tightly coupled with type inference.

Type Inference: defines the `infer` predicate, which attempts to deduce the type of a term in a given context from its structure. This is the core of CoC type-checking.

After completing my implementation, I tested it against a handful of test programs featuring common patterns in DTT, which provides comprehensive coverage of standard CoC features. These test programs can be found here. My typechecker was successful at type-checking each of these programs, and moreover is also able to catch typing errors in input programs that are not well-typed. Despite lacking a specialized type-theoretic unification algorithm, my CoC system can handle many common declaration patterns involving universe polymorphism or complex term dependencies. Simply replacing type and universe parameters with a fresh Epilog variable and allowing the native unification process of Epilog to find a ground term that solves for that variable was successful at resolving most terms of polymorphic declarations. Moreover, when compared to similar CoC typecheckers written in more traditional programming paradigms, such as Typechecker Zoo’s Rust implementation [12], my design is significantly more simple. My project required a fraction of the infrastructure required to support the same language features, and I was able to minimal boilerplate by leveraging techniques innate to logic programming such as pattern-matching on the structure of compound terms within predicates.

5 Challenges

Naturally, porting a CoC implementation to Epilog was also accompanied by its own challenges and obstacles. Debugging in general was difficult, due to the tight coupling of disparate components of the CoC ecosystem. In many cases it was not possible to isolate an individual predicate for unit testing, since the term-level typechecking predicate `check` and declaration-level typechecking predicate `check_declaration` both involve

calls to the highly recursive `infer` rule. Identifying problems in the system often involved manually scanning several hundred lines of resolution traces.

While most of the infrastructure of a typical CoC system fell into place naturally, there were also a few boundary cases that required special care to embed in logic programming. Firstly, pattern matching in a functional language is exclusive; an expression is matched against at most one constructor. In many cases, certain functions that recurse on the structure of a term only descend to the children of particular constructors, such as the `check` predicate, which only does something interesting for λ -abstraction terms. Inputs which don't match the patterns of interest (and *only* these inputs!) should be matched against a wildcard and handled by a default case.

On the other hand, in Epilog, the unifier attempts to match a term against all available rule definitions. This meant that for any rule with a wildcard pattern, additional clauses had to be added to make sure no other pattern was already matched. Otherwise, the Epilog interpreter could “cheat” by unifying a variable with an expression matching one of that rule's non-default patterns, but still fall through to the default case. This caused a handful of subtle bugs that were difficult to trace.

Another difficulty presented by the Epilog implementation originated from the interpreter's preference for depth-first search during variable resolution. In particular, when performing α -conversion with the `alpha_normalize` predicate on a term containing a metavariable, the unifier would become stuck trying to expand the first binary constructor it encountered in the rule's definition. Since terms must be converted to de Bruijn form using `alpha_normalize` before they can be unified by Epilog (as their ground components must be *syntactically* identical for resolution to succeed), I needed to find a way to implement `alpha_normalize` for terms potentially carrying pre-unified metavariables. This would require somehow forcing the interpreter to expand rule definitions breadth-first, to guarantee termination.

My solution to this problem was inelegant, but in practice it achieved the desired outcome. I modified `alpha_normalize` so that it accumulates an extra `Depth` parameter, which increases each time the interpreter descends on the children of a binary constructor. This is checked against a constant `max_depth` that the ruleset stores as a fixed rule (this could equivalently be an element of the underlying dataset), and if the maximum depth is exceeded by the `Depth` parameter the current branch fails. This forces Epilog to explore the remaining definitions, to avoid getting stuck expanding a single constructor forever. In theory this solution also invalidates typechecking certain programs, since they may require an α -conversion exceeding the arbitrarily chosen maximum depth, but for most programs that occur in nature a maximum depth of no more than around three or four suffices.

6 Future Work

Although my CoC implementation successfully type-checks most well-typed declarations, it currently cannot resolve the types of instantiations of struct and inductive declarations with universe parameters under specific conditions. In particular, if a struct or inductive type takes one or more universe parameters that do not occur in a particular field or constructor (respectively), then defining a term using that field projection or pattern matching against that constructor causes the typechecker to time out. I suspect the reason for this is that the unused universe parameters are replaced with metavariables that cannot be unified, so the typechecker cannot resolve the term's universe. I aim to fix this bug, but have not yet been able to find a satisfactory solution.

Another important feature lacking from my current design which I hope to introduce in the future is propositional equality. This is needed to write many of the kinds of proofs one would expect a typical proof assistant to support. The main obstruction to propositional equality types in my current design is that the `def_eq` predicate, which checks whether a pair of terms are definitionally equal (up to normalization) in a given context, is too weak to deduce all of the equality judgments needed to properly implement an ergonomic reflexivity introduction rule. With some improvements to `def_eq` this should become doable, but this particular predicate is surprisingly fragile; during development most of the attempts at generalizing definitional equality further caused some other component of the system to fail.

References

- [1] Carlo Angiuli and Daniel Gratzer. *Principles of Dependent Type Theory*. <https://www.danielgratzer.com/papers/type-theory-book.pdf>. Mar. 2026.
- [2] Gilles Dowek. “The undecidability of typability in the Lambda-Pi-calculus”. In: *Typed Lambda Calculi and Applications*. Ed. by Marc Bezem and Jan Friso Groote. Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 139–145. ISBN: 978-3-540-47586-6.
- [3] *EpilogJS*. <http://epilog.stanford.edu/documentation/epilogjs/guide.php>.
- [4] Martín Hötzel Escardó. *Introduction to Univalent Foundations of Mathematics with Agda*. 2022. arXiv: 1911.00580 [cs.LO]. URL: <https://arxiv.org/abs/1911.00580>.
- [5] William A. Howard. “The formulae-as-types notion of construction”. In: 1969. URL: <https://api.semanticscholar.org/CorpusID:118720122>.
- [6] *Logic Programming*. <http://logicprogramming.stanford.edu/public/index.php>.
- [7] nLab authors. *type theory*. <https://ncatlab.org/nlab/show/type+theory>. Revision 150. June 2026.
- [8] Christine Paulin-Mohring. “Introduction to the Calculus of Inductive Constructions”. In: *All about Proofs, Proofs for All*. Ed. by Bruno Woltzenlogel Paleo and David Delahaye. Vol. 55. Studies in Logic (Mathematical logic and foundations). College Publications, Jan. 2015. URL: <https://inria.hal.science/hal-01094195>.
- [9] Benjamin C. Pierce. *Types and Programming Languages*. The MIT Press, Jan. 2002. ISBN: 978-0262162098.
- [10] Xichen Tang. *A Comprehensive Survey of the Lean 4 Theorem Prover: Architecture, Applications, and Advances*. 2025. arXiv: 2501.18639 [cs.LO]. URL: <https://arxiv.org/abs/2501.18639>.
- [11] *The Rocq Prover*. <https://rocq-prover.org/about>.
- [12] *Typechecker Zoo: Calculus of Constructions*. <https://sdiehl.github.io/typechecker-zoo/coc/calculus-of-constructions.html>.